

Bookstore App with MongoDB & Azure Kubernetes Service (AKS) - Option 2

Course: AIN-3003 - Database Systems and Cloud Computing

Instructor: Gökşin Bakır

Submission Date: January 7, 2025

Student Name: Abdullah Hani Abdellatif Al-Shobaki

Student Number: 2284612

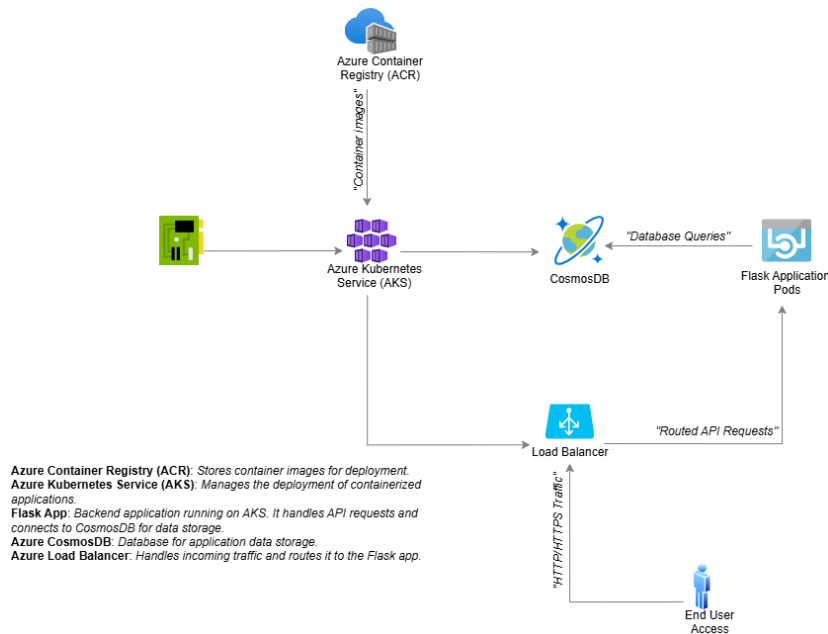
Table of Contents

- 1. Project Overview
- 2. Project Architecture
- 3. Pre-requisites
- 4. Deployment Steps
 - 4.1. Azure CosmosDB Setup
 - 4.2. Docker Image Creation
 - 4.3. AKS Cluster Setup & Deployment
 - 4.4. YAML Files Overview
- 5. Application Features & CRUD Operations
- 6. Service Discovery & Networking
- 7. YAML Configuration Files
- 8. Testing & Validation
- 9. Challenges & Solutions
- 10. Future Improvements

1. Project Overview

This project will containerize a Python application using Flask and MongoDB to be deployed on Azure Kubernetes Service (AKS). The project will show how to deploy scalable apps, securely stored data, and service orchestration using cloud-native services.

2. Project Architecture



Above is the architecture which illustrates the integration of Azure Kubernetes Service (AKS), Flask application pods, and MongoDB CosmosDB, with Docker images stored in Azure Container Registry and network services managed via Kubernetes LoadBalancer and ClusterIP.

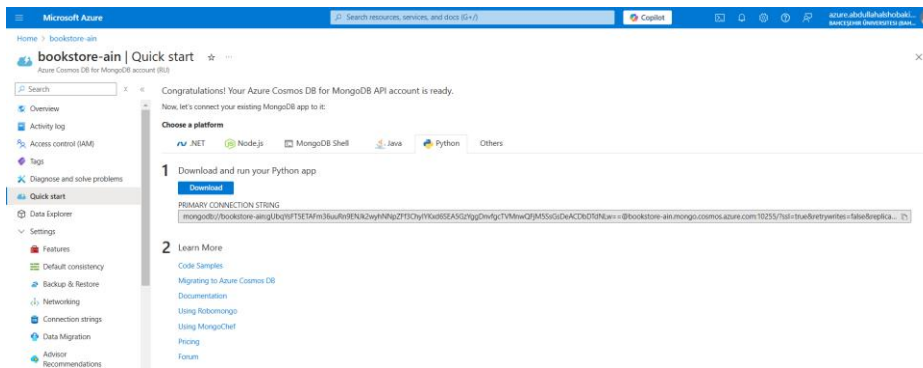
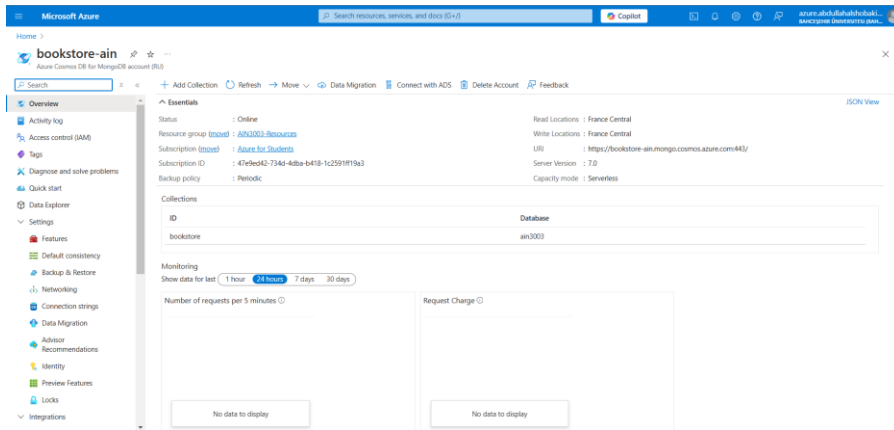
3. Pre-requisites

- Azure Subscription
- Azure CLI Installed
- Docker Installed
- kubectl Installed
- Python 3.x and Flask for Local Testing

4. Deployment Steps

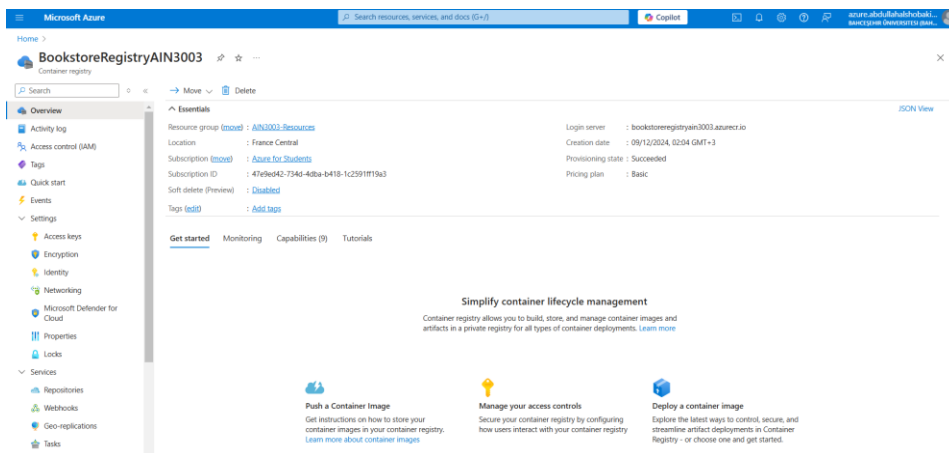
4.1. Azure CosmosDB Setup

1. Log in to Azure Portal and create a CosmosDB MongoDB API instance.
2. Enable Public Access (for testing only).
3. Collect Primary Connection String from Azure CosmosDB.



4.2. Azure Container Registry (ACR) Setup

1. Set Up ACR via Azure Portal
2. Retrieve the ACR Login Server URL from the ACR Overview Page.



4.3. Docker Image Creation

1. Login to ACR:

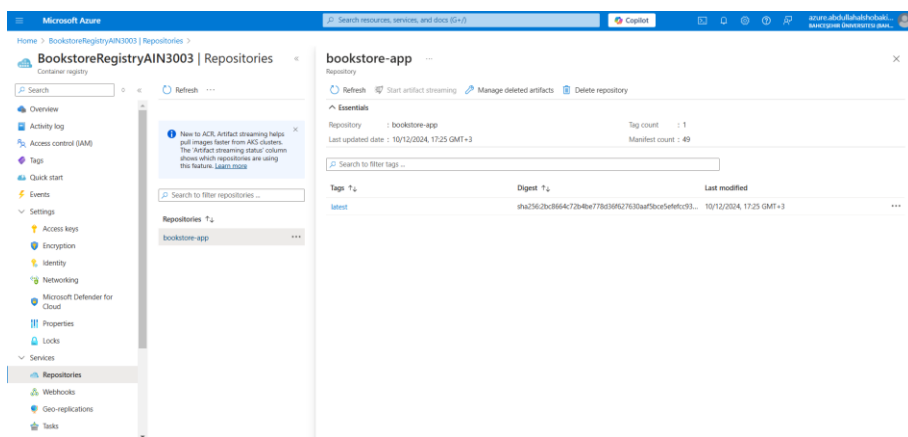
```
az acr login --name BookstoreRegistryAIN3003
```

2. Build the Docker Image:

```
docker build -t bookstoreregistryain3003.azurecr.io/bookstore-app:latest.
```

3. Push the Image to ACR:

```
docker push bookstoreregistryain3003.azurecr.io/bookstore-app:latest
```

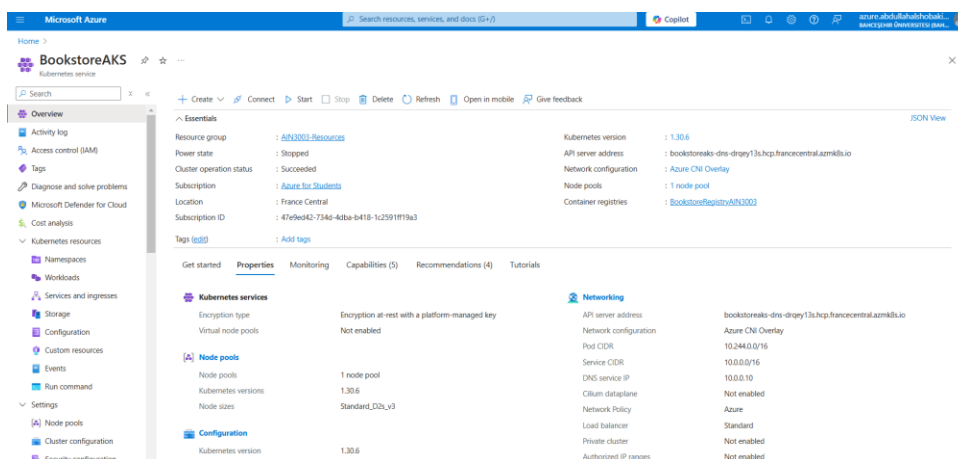


4.4. Azure Kubernetes Service (AKS) Setup

1. Create an AKS Cluster.

2. Connect the AKS Cluster to ACR:

- On the AKS cluster configuration page, under Settings > Integrations, link the ACR instance.



4.5. Kubernetes Application Deployment

1. Deploy YAML Files in Order:

- Persistent Volume Claim:

```
kubectl apply -f YAML/mongodb-pvc.yaml
```

- MongoDB Deployment and Service:

```
kubectl apply -f YAML/mongodb-deployment.yaml
```

```
kubectl apply -f YAML/mongodb-service.yaml
```

- Flask App Deployment and Service:

```
kubectl apply -f YAML/deployment.yaml
```

```
kubectl apply -f YAML/service.yaml
```

- Apply ConfigMaps and Secrets:

```
kubectl apply -f YAML/configmap.yaml
```

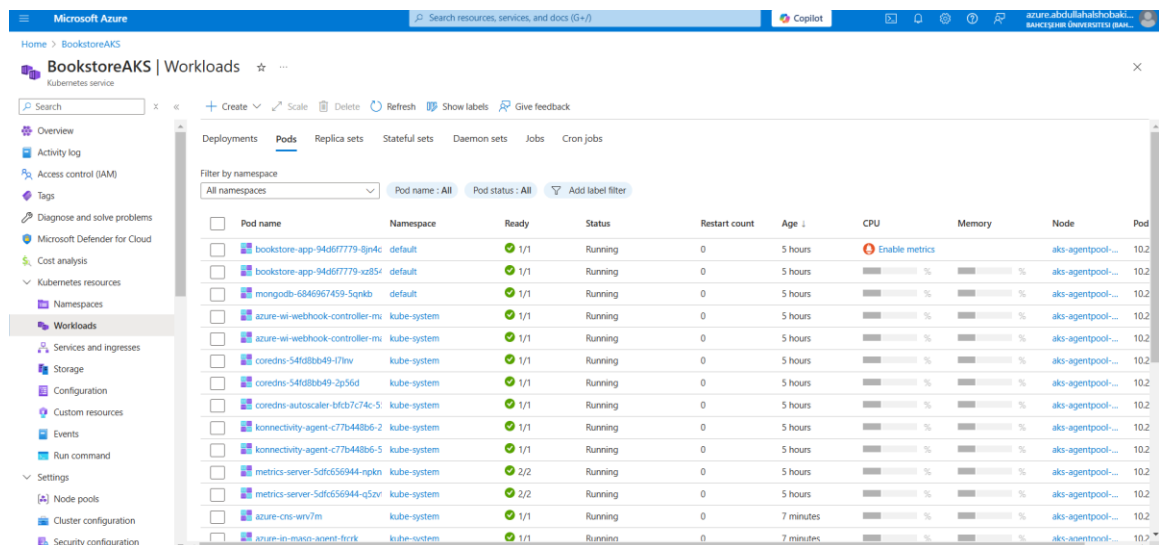
```
kubectl apply -f YAML/secrets.yaml
```

- Apply Network Policies:

```
kubectl apply -f YAML/network-policy.yaml
```

2. Check Application Status:

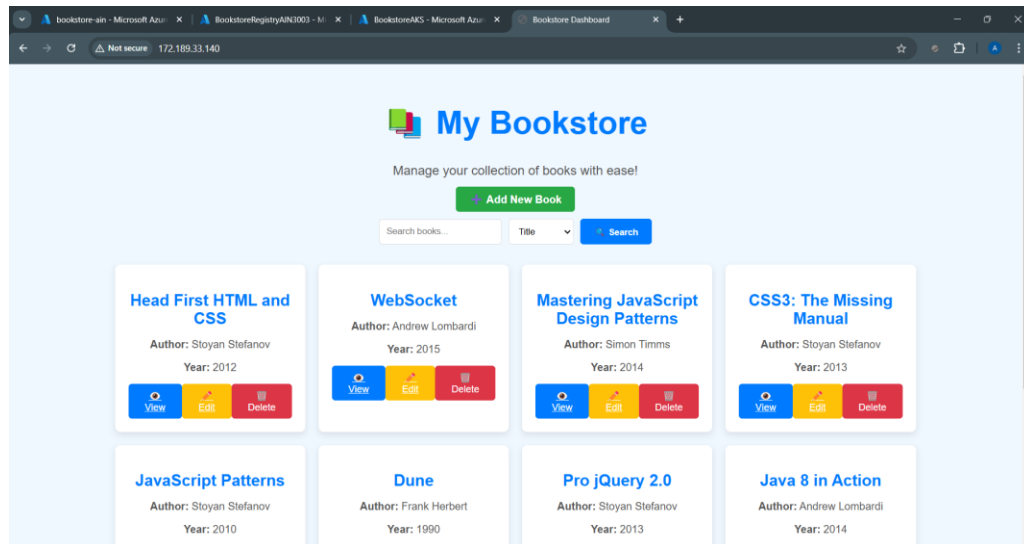
```
kubectl get all
```



Pod name	Namespace	Ready	Status	Restart count	Age	CPU	Memory	Node	Pod
bookstore-app-94d6f779-8jv4c	default	1/1	Running	0	5 hours	Enable metrics		aks-agentpool...	10.2
bookstore-app-94d6f779-nc854	default	1/1	Running	0	5 hours			aks-agentpool...	10.2
mongodb-6846967459-5qknb	default	1/1	Running	0	5 hours			aks-agentpool...	10.2
azure-wi-webhook-controller-mi	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
azure-wi-webhook-controller-mi	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
coredns-54f8bb49-47aw	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
coredns-54f8bb49-2p56d	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
coredns-autoscaler-bfcb7c74c-5	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
konnectivity-agent-c77b448b6-2	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
konnectivity-agent-c77b448b6-5	kube-system	1/1	Running	0	5 hours			aks-agentpool...	10.2
metrics-server-5dfc656944-npkn	kube-system	2/2	Running	0	5 hours			aks-agentpool...	10.2
metrics-server-5dfc656944-q5zv	kube-system	2/2	Running	0	5 hours			aks-agentpool...	10.2
azure-cns-wm7m	kube-system	1/1	Running	0	7 minutes			aks-agentpool...	10.2
azure-in-mono-agent-frick	kube-system	1/1	Running	0	7 minutes			aks-agentpool...	10.2

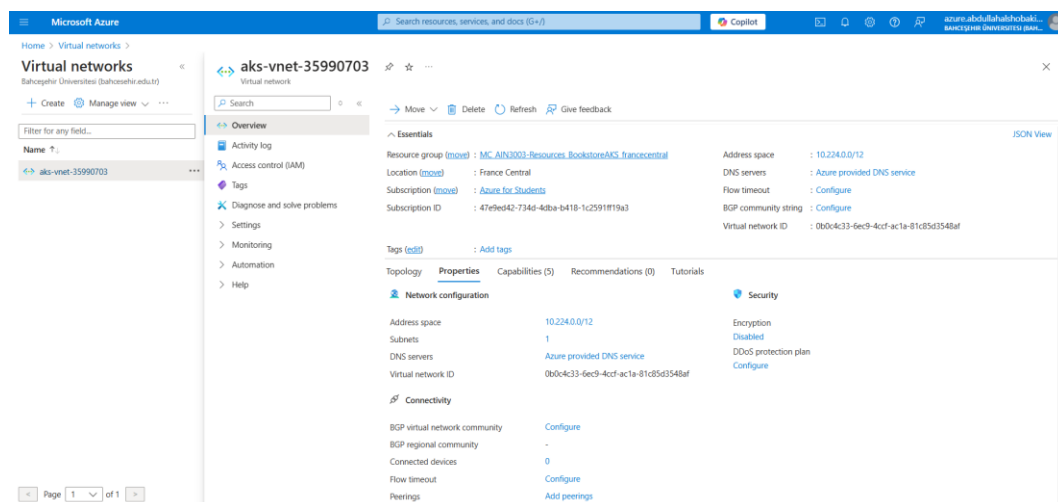
5. Application Features & CRUD Operations

- Create: Add a new book
- Read: Display all books
- Update: Edit book details
- Delete: Remove a book



6. Service Discovery & Networking

- The AKS cluster is deployed in an Azure Virtual Network (VNet), named `aks-vnet-35990703`, to ensure secure communication between services.
- Kubernetes Network Policies are implemented to prevent unauthorized access and ensure secure communication in the cluster.
- The Flask application uses a Kubernetes ClusterIP service to communicate with MongoDB for internal service discovery.



7. YAML Configuration Files

1. ConfigMap YAML
 - Purpose: Stores environment-specific configuration values for the Flask application, such as environment variables.
 - File Location: YAML/configmap.yaml
2. Flask Application Deployment YAML
 - Purpose: Deploys the Flask application pods to the Kubernetes cluster.
 - File Location: YAML/deployment.yaml
3. MongoDB Deployment YAML
 - Purpose: Deploys the MongoDB database instance within the Kubernetes cluster.
 - File Location: YAML/mongodb-deployment.yaml
4. MongoDB Service YAML
 - Purpose: Exposes the MongoDB database inside the cluster, so that other services can communicate with it.
 - File Location: YAML/mongodb-service.yaml
5. Flask Application Service YAML
 - Purpose: Exposes the Flask application so that it can be accessed externally from the outside world through a LoadBalancer.
 - File Location: YAML/service.yaml
6. Network Policy YAML
 - Purpose: Restricts traffic between pods and secures communication within the cluster.
 - File Location: YAML/network-policy.yaml
7. Persistent Volume Claim (PVC) YAML
 - Purpose: Requests persistent storage for MongoDB data, ensuring data is not lost if a pod restarts.
 - File Location: YAML/mongodb-pvc.yaml
8. Secrets YAML
 - Purpose: Stores sensitive data, such as database credentials, securely within the Kubernetes cluster.
 - File Location: YAML/secrets.yaml

8. Testing & Validation

- App Access: The application is accessible via the LoadBalancer URL: <http://172.189.33.140> (*Works only when the AKS cluster is running.*)
- Successfully tested the following CRUD Operations:
 1. Create: Adding new books.
 2. Read: Viewing and searching for books.
 3. Update: Editing book details.
 4. Delete: Removing books from the collection.
- A video showcasing all CRUD operations and the app's functionality has been recorded.

9. Challenges & Solutions

1. Missing Database and Collection

- Problem: CosmosDB did not have the `bookstore` database and the `books` collection.
- Solution: The database and the collection had to be manually created using Azure Portal.

2. Azure - Insufficient vCPU quota

- Problem: Azure student subscription did not have any available vCPUs in the France Central region.
- Solution: Registered Microsoft.Compute, Microsoft.ContainerService, Microsoft.Network, and Microsoft.Storage services.

3. Flask App Connection Failure

- Problem: Flask could not connect to MongoDB due to incorrect MongoDB URI or network policy restrictions.
- Solution: Corrected MongoDB URI, updated service names, and fixed network policies.

4. Internal Server Error (500)

- Problem: The Flask app returned a 500 Internal Server Error due to failure in connecting to the database.
- Solution: Fixed service configurations and updated environment variables.

10. Future Improvements

- Enhanced Security: Use Azure Active Directory (Azure AD).
- Monitoring & Alerts: Add Azure Monitoring with custom alerts.
- CI/CD Integration: Set up GitHub Actions for automatic deployment.